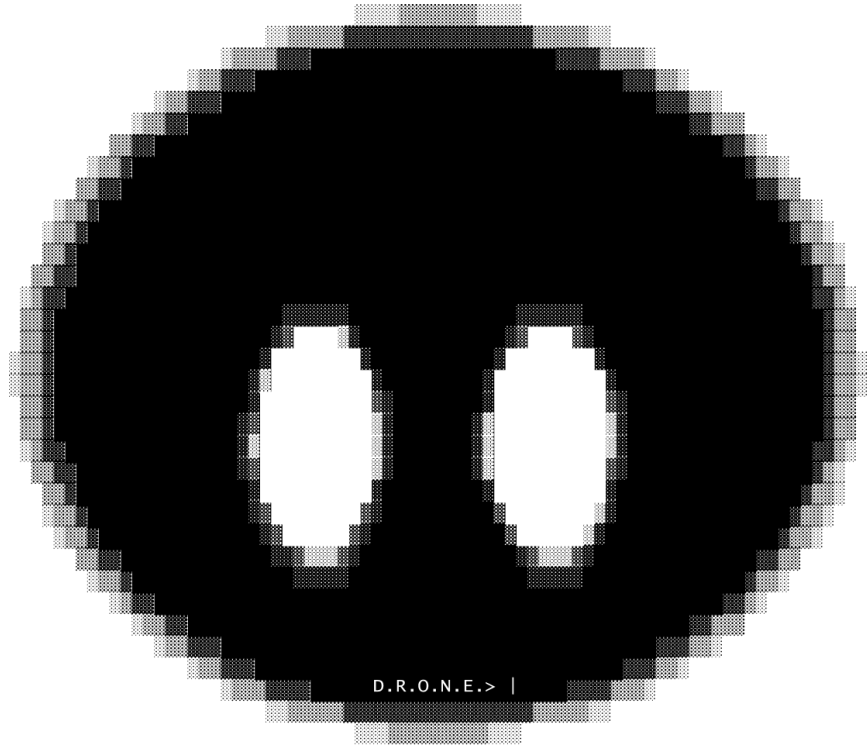




Dynamic Responsive Optimized Neural Engine

How a small AI written from scratch in C thinks, works, speaks and learns

Maximum Tension™ - Teoman Deniz - 2026/10/07

Abstract

D.R.O.N.E. (Dynamic Responsive Optimized Neural Engine) is a small artificial intelligence written from scratch in C. It uses no outside libraries and no pre-trained model, and it runs offline on an ordinary CPU. Each AI built on the engine is called a *drone*; the first one is called Lucy.

The design splits the work of answering into two parts. An exact **core**, written as ordinary C, reads every message, keeps track of the conversation and decides what should happen: run an action, state a thought, or just talk. Every action becomes a **task** that the core checks after it runs; when it did not happen, the core finds the cause and tries the next fix, until it is done or no fix is left. A **neural brain** then puts the result into words. Its neurons are small pieces of x86-64 machine code that the engine writes at run time. Between the two parts, a set of engine checks keeps the reply honest: it must match what was really done and computed.

What the AI can do is not built into the engine. **Skills** are plug-in libraries that come with plain-text *know-how*: what each action makes true, how to check it and what to try when it fails. The sentences the AI says on its own, the ways it understands "it is still there" or "try again", and what it knows about the world all live in plain-text files that can be changed while it runs.

This paper explains how each part works and how to use it. Every mechanism is shown with a real example taken from the running system, and the mathematics is given where it is used. Section 4 follows one message through the whole engine, step by step, section 9 shows how the core keeps at a task, and section 15 shows how to build, run, extend and train it. The numbers are a snapshot from October 7, 2026.

Table 1. Key facts about the engine.

At a glance	
Language	C, about 150,000 lines for the engine and its two skills. Python tools for training data and checks, about 5,800 lines.
Outside libraries	None. Only the C standard library and the operating system are used. Skills and the GPU driver are loaded at run time.
Hardware	Any x86-64 CPU with AVX2 and FMA (Intel Core from 2013 on). An optional Vulkan GPU path is in progress.
Brain	11.4 million parameters and 26,255 living neurons. 4 layers of width 256, 16 attention heads per layer, 17,359 word neurons.
Near memory	512 tokens, taken from the last 64 messages.
Skills	Plug-in libraries (.dll on Windows, .so on Linux) with plain-text know-how: computer (files, folders, the trash, the clock) and reason (puzzles with one right answer).
Platforms	Windows (MinGW) and Linux. A brain trained on one runs on the other unchanged.
Tests	1,256 sense-core checks, 99 knowledge-core checks, 104 whole-conversation checks, nine honesty suites and the engine proofs. Two held-out skill tests of 2,302 and 1,860 turns.

Contents

1. Why D.R.O.N.E. Exists	4
2. Design Principles	4
3. System Overview	5
4. A Worked Example, End to End	6
5. The Engine: Neurons as Machine Code	8
6. The Neural Brain: the Voice	10
7. Training	12
8. The Sense Core: Understanding and Deciding	15
9. Work: Doing It, Checking It, Fixing It	18
10. Skills, Packages and Sayings.....	21
11. Speaking: From Decision to Words.....	22
12. Memory and Knowledge	24
13. Testing and Honesty	25
14. Results	26
15. Using and Extending the Engine	28
16. Roadmap	31
17. References.....	32
Appendix A. Glossary.....	32

1. Why D.R.O.N.E. Exists

1.1 Goals

- **Correct answers from minimum knowledge.** The AI should be right because it reasons and computes, not because it memorized the answer.
- **Honest by construction.** It never claims an action that did not happen. It never states math that the calculator did not confirm. When it does not know something, it says so.
- **Keeps at it.** Every action is checked after it runs. When something did not work, the AI looks for the cause and tries another way, and it never offers the same thing twice.
- **Small and offline.** It runs on one ordinary computer, with no internet and no GPU needed.
- **Understandable.** Every decision can be shown: thought lines in the chat, and the `/status`, `/context`, `/mind`, `/brain` and `/neuron` commands.
- **Growable.** The brain grows while it trains. New kinds of neurons, skills, knowledge packages and sentences can be added without starting over.
- **One engine, many drones.** In the long term the same engine should power an assistant, a character (NPC) in a game, and a worker that drives a body: a *Worker D.R.O.N.E.*

1.2 What It Is Not

D.R.O.N.E. is not a large language model, and it is not a wrapper around one. It has no pre-trained weights (yet), it does not browse the internet (yet), and it does not send anything off the computer. Everything it knows comes from its training data, its knowledge files and what the user tells it.

2. Design Principles

A few rules shape every part of the engine. The most important one came from the calculator. Once math was done by exact code instead of by the neurons, wrong math disappeared. The same idea was then used everywhere.

Table 2. The design principles of D.R.O.N.E.

Principle	What it means in practice
We write mechanisms, never answers	C code may contain <i>how</i> to parse, compute, compare, plan and check. It never contains facts ("a cat has 4 legs") or reply sentences.
Exact where possible, learned where not	Math, counting, spelling, dates, file actions and memory lookups are exact code. Wording, tone and fluent replies are learned.
The core decides, the voice speaks	The neural brain does not choose actions on its own. It puts the core's decision into words.
Check, never assume	After every action the core checks that it really happened. "Done" is only said when the check agrees.
Checks live in the engine	Honesty and copy checks run outside the brain, so they work with any trained brain, old or new.
Skills and sentences live in files	What a skill can do and how to fix it, the sentences the AI says on its own and the ways it reads the user are plain lines, read again when they change.
Every change behind a test	Each fix adds regression checks. A training round keeps a new brain only when its held-out scores hold up.

Principle	What it means in practice
Readable for people	Thought lines are plain text. The chat layout leaves space between messages so it is easier to read.

3. System Overview

Figure 1 shows the path of one message through the engine, from the user's words to the reply.

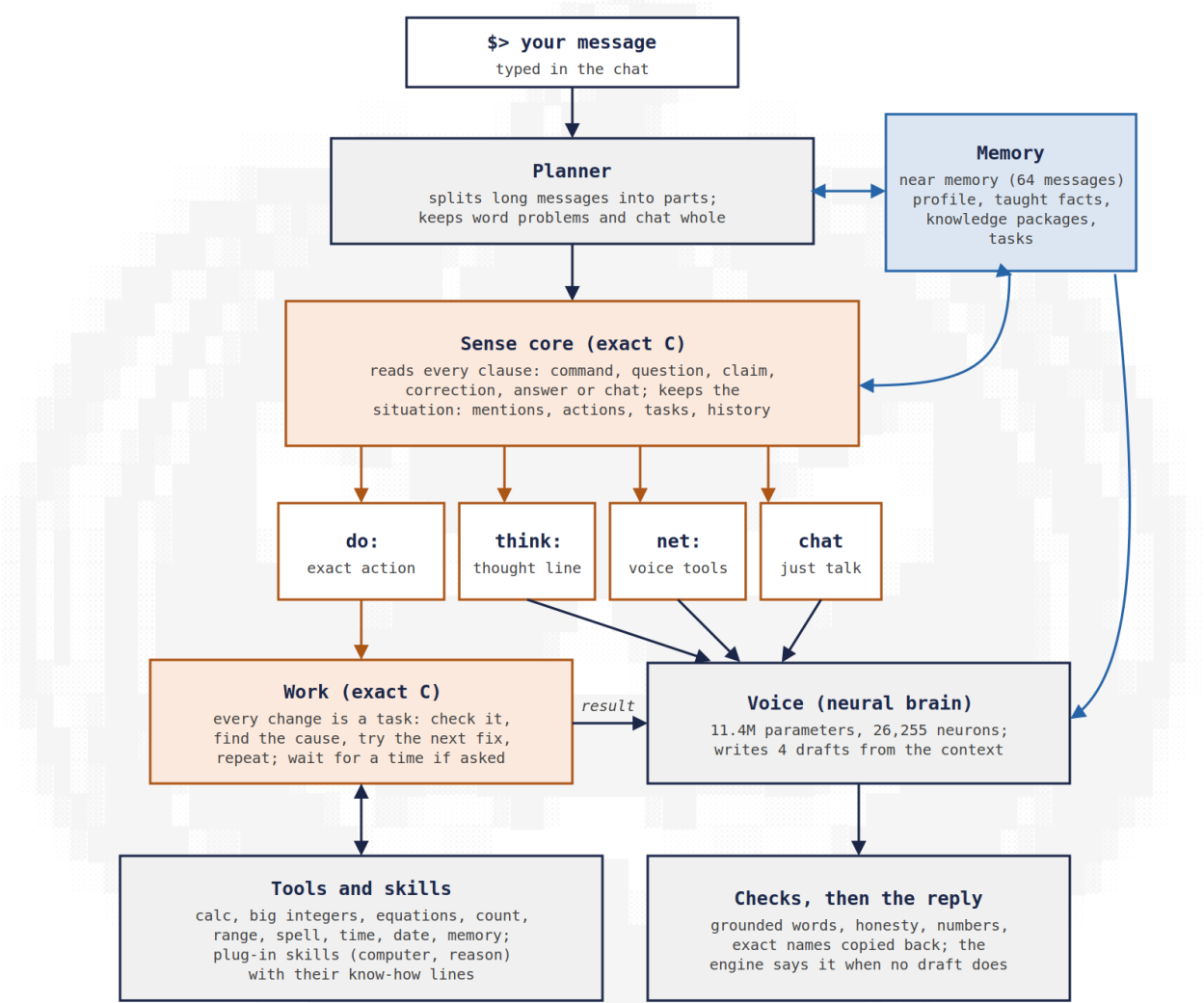


Figure 1. The path of one message through D.R.O.N.E. The exact parts are orange, the neural part is gray and memory is blue.

The **planner** first splits a long message into parts at sentence ends and at words such as "and then" or "lastly". It keeps word problems, quoted text and pure chit-chat whole. Next, the **sense core** reads each part and decides one of four things:

- **do:** run an exact action, such as `calc`, `write` or `move`; every change becomes a task for the work core;
- **think:** state a thought, such as "you did not tell me your age" or "I did not send an email";
- **net:** let the brain use its own tools, which must be justified by the user's words;
- **chat:** just talk.

The **work core** runs each task, checks it and fixes what went wrong (section 9). Tools and skills return exact results. Those results, the thought lines and the recent conversation form the **context** that the neural brain reads. The brain writes four drafts, the engine checks and scores them, and one of the best becomes the reply. When no draft says what the core found, the engine says it in its own sentence.

One program runs several **agents** on their own threads. The input agent reads what the user types at any time, even while the AI is answering. The conversation agent answers one message at a time and checks the mailbox between the parts of a long answer, so "stop" ends it at once and "also ..." joins it. Job agents do long work in the background, such as studying a file sentence by sentence.

In the chat the user types after `$>`. The reply line starts with the drone's name, which comes from `mind_original/self.txt`; the examples in this paper use the name *Drone*. Lines in brackets show what the engine did or thought before answering:

```
$> spell the word drone

[spell drone -> D-R-O-N-E]
Drone> D-R-O-N-E

$>
```

Example 1. The chat layout. The bracket line is the core's thought; the reply is the voice's.

4. A Worked Example, End to End

The quickest way to see how the parts fit together is to follow one real message. Everything in this section was produced by the running engine. Only line breaks were changed.

```
$> I had 20 candies, gave 5 to my brother and ate 3. How many are left?

[calc 20 - 5 - 3 -> 12]
Drone> 12 candies in total.
```

Example 2. A word problem. The number comes from the calculator; the wording ("in total" instead of "left") is the voice's.

4.1 Step by Step

Table 3. The nine stages of Example 2.

#	Stage	What happens	Result
1	Planner	The message asks "how many ... left" and holds three numbers, so it is a word problem and is kept whole instead of being split at the full stop.	one part
2	Words	The core reads numbers (20, 5, 3), verbs ("had", "gave", "ate") and the question words.	tokens
3	Clauses and kind	"I had 20 candies" and "gave 5 ... ate 3" are past-tense statements, so they trigger no action. "How many are left?" is a question.	question
4	Word problem solver	"gave" and "ate" are subtract cues. All the operators are of one kind, so the reading is safe.	<code>calc 20 - 5 - 3</code>
5	Tool	The calculator evaluates the expression exactly.	<code>12</code>
6	Context	Memory lines, the message, the action and its result are written as tokens for the brain (4.2).	66 tokens

#	Stage	What happens	Result
7	Voice	The brain writes four drafts.	4 drafts
8	Checks and pick	Every draft states 12, so none is penalized. One is drawn by score (4.3).	one reply
9	Copy-back	The reply's number already matches the result, so nothing needs repairing.	final reply

4.2 What the Brain Reads

The `/context` command prints exactly what the brain sees. Each line starts with a role token: `<mem>` for memory, `<u>` for the user, `<do>` and `<res>` for an action and its result, and `<c>` for the AI's own turn. The words are lowercased word neurons:

```
<mem> my name is drone .
<mem> i am a small ai . i run offline on this computer .
<mem> i was made on september 25 , 2026 .
<u> i had 20 candies , gave 5 to my brother and ate 3 . how many are left ?
    <do> calc 20 - 5 - 3 <res> 12
<c> 12 candies in total .
```

Example 3. The context of Example 2 (66 tokens), printed by `/context` after the reply.

The brain never computes $20 - 5 - 3$ itself. It only has to notice the `<res> 12` and say it. This is the core idea of the whole system: the hard, exact part is done by code, and the network learns the much easier job of reporting it.

4.3 Drafts, Scores and the Pick

The `/status` command lists the drafts with their scores, computed with equation (11) in section 11:

Table 4. The four drafts of Example 2, from `/status`.

Draft	Score S	Drafts like it
12 candies in total.	-0.31	3 of 4
It is 12.	-0.35	1 of 4

The pick is a softmax over the scores with $\tau = 0.15$ (equation 12). The three identical drafts together get

$$P = \frac{3e^{-0.31/0.15}}{3e^{-0.31/0.15} + e^{-0.35/0.15}} = \frac{3}{3 + e^{-0.04/0.15}} = \frac{3}{3 + 0.766} \approx 0.80$$

and "It is 12." gets about 0.20. Both are correct, so the softmax only adds variety. A draft that broke an honesty rule would sit about 8 points lower, and its share would be about $\exp(-8 / 0.15) \approx 10^{-23}$, which in practice is never.

4.4 A Task With a Hidden Step

The second example shows the planner, the situation model and the work core together:

```
$> Can you create notes.txt and then move it into a folder called work?

[plan] 2 parts: (1) Can you create notes.txt (2) move it into a folder called work?
[write notes.txt -> written]
Drone> Done, I made the file notes.txt.
[move notes.txt to work -> no such folder]
[mkdir work -> folder made]
[check: exists work -> folder: done]
[move notes.txt to work -> moved]
[check: exists notes.txt -> no: done]
Drone> Done, I made the folder work. I also moved notes.txt to work.
```

Example 4. A two-part task. The folder did not exist, so the core made it, checked it, and moved the file again.

- The planner splits at "and then", so each part is decided and answered on its own.
- In part 2, "it" is resolved by the situation model: the last file mentioned and made was notes.txt.
- The move fails with `no such folder`. One fix is allowed for that: make the missing folder, but only if the user named it ("a folder called work"). The new folder is checked, then the move is done again and checked too: a moved file is no longer where it was. The engine never invents a folder name, and a missing *file* is never "fixed" by making a folder.
- The reply tells what was done. The first move is not mentioned, because a later step did it.

4.5 When the Voice Disagrees With the Core

The checks matter most when the brain gets it wrong. Asked "did you order a pizza?", the core decides `did order pizza -> no`. A small brain trained partly on human chat is tempted to agree ("Of course."), so any draft whose yes or no contradicts the core loses 8 points. For a "no" of this kind (I didn't, I can't, I don't have one), the engine goes further and states the answer itself, with the reason taken from what the AI knows about itself:

```
$> did you order a pizza?

[did order pizza -> no: I run offline on this computer]
Drone> No, I didn't. I run offline on this computer.
```

Example 5. The answer and its reason come from the core, so no draft can turn it into a yes.

5. The Engine: Neurons as Machine Code

Most neural networks are big matrices processed by a library. In D.R.O.N.E. every neuron is its own small piece of x86-64 machine code. The engine has its own assembler, which covers general-purpose instructions and the VEX-encoded AVX2 and FMA vector instructions [22]. It writes this code into executable memory while the program runs.

5.1 One Neuron, One Stub

A neuron is a *stub* of about 30 to 41 bytes. The stub does not do the math itself. It loads the address of its own weights, records where its activations live, sets the address to continue at, and jumps into an **organ**. An organ is shared AVX2/FMA code that does the math for every neuron of the same kind in the same area. When the organ finishes, it jumps to the continuation address, which is the next neuron's stub. The whole forward pass is a single chain of jumps, with no loops over a neuron list and no function calls.

```

; neuron #1041 (ffn, area 1), forward role. Offsets are illustrative.
lea  r12, [r13 + 0x0004A2C0]    ; r12 = this neuron's weights
mov  qword [r15 + na], 0x1F400 ; tell the organ where the activations live
lea  rbp, [rip + next]         ; rbp = where to continue: the next stub
jmp  organ_ffn_area1_fwd      ; shared AVX2/FMA code does the math

```

Example 6. The four instructions of a neuron stub (from `GENERATE_STUB` in `KERNEL.c`).

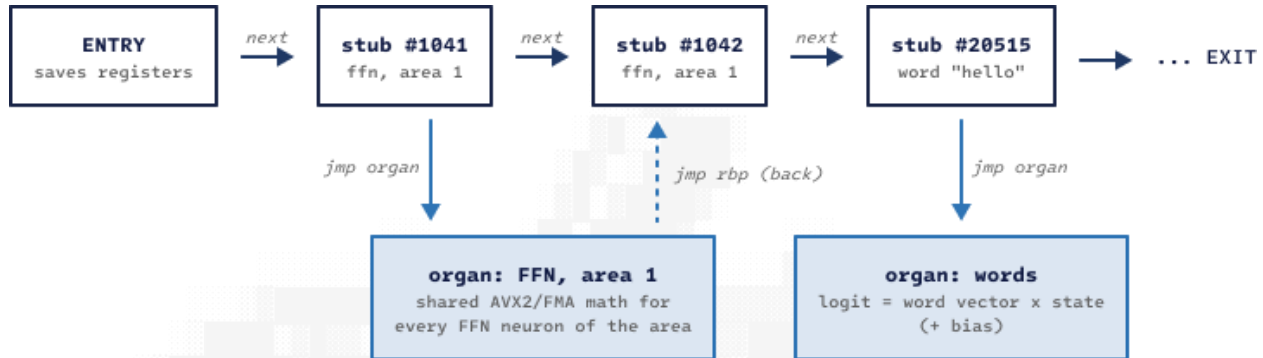


Figure 2. Neurons are stubs chained by jumps. Organs hold the shared math and jump back through `rbp`.

A fixed register convention makes the stubs this short:

Table 5. Register convention shared by every stub and organ.

Register	Holds
<code>r15</code>	Ctx: the run context (mode, sizes, learning rate, scratch constants)
<code>r14</code>	Activation base: all values computed in the forward pass
<code>r13</code>	Weight base: the start of the parameter plane
<code>r12</code>	This neuron's own weights (set by the stub)
<code>rbp</code>	Continuation: the address the organ jumps to when it is done
<code>rbx</code>	Function table back into C (READ, WRITE, CREATE, CONNECT, TRACE)

5.2 Chains

One call from C enters the chain at the ENTRY stub, and one `ret` leaves it at the EXIT stub. In between, the chain runs the forward pass (embedding, then for each area a norm and its neurons, then the word neurons and the copy neuron). After that it reaches a GATE that exits when only inference is needed, then the LOSS, then a second GATE that exits for evaluation, and then the backward pass in reverse order. A separate update chain holds one Adam block per neuron. Its last element is a GROW neuron, which can ask C to CREATE new neurons, so the network grows itself.

5.3 Memory Layout

Code lives in 64 KB segments of executable memory. On systems that enforce W^X , pages are writable while code is generated and executable while it runs, never both. Weights live in four *planes* that sit a fixed distance apart (PLANE = 256 MB). So one displacement reaches a parameter, its gradient and both Adam moments:

```

weight base r13
|
+--> plane p : parameters      [r13 + off]
+--> plane g : gradients       [r13 + off + 1 x PLANE]
+--> plane m : Adam 1st moment [r13 + off + 2 x PLANE]
+--> plane v : Adam 2nd moment [r13 + off + 3 x PLANE]      PLANE = 256 MB

```

Example 7. Weight planes. The code uses only base + displacement, so it is reentrant.

Because the code only uses base plus displacement, it is reentrant. Training threads run the very same machine code, each with its own context, activations and weight copy. The brain file stores neurons, code pieces with a list of jump targets, and fp16 weights. Loading means copying the bytes and patching the jumps. Saving and loading are bit-identical. In *play mode* (`--no-learn`) the weights stay in 16-bit form in RAM. This is the fastest mode and uses the least memory, which suits game characters.

5.4 Recipe Neurons

New kinds of neurons can be described in a small text file called a *recipe*. The engine checks the recipe, works out its backward (learning) step by itself, and compiles both to machine code. The recipe is stored inside the brain file, so a brain that uses it still works on any machine. The proofs compare every recipe against plain C and against finite differences.

```

recipe gated
param wg 64 d read      # small random start
param wu 64 d read
param wd 64 d write     # starts silent when added to a trained brain
x = in
g = mv wg x
u = mv wu x
s = silu g
h = mul s u
y = mvt wd h
out y

```

Example 8. A gated FFN neuron written as a recipe (recipes/gated.rcp).

5.5 GPU Groundwork

A Vulkan backend is in progress. The engine loads the driver at run time, so no SDK is needed. It writes its own GPU programs as SPIR-V [23], and all of them pass the official validator. On the user's RTX 4050 laptop GPU, a tiled matrix multiply reached 1,058 GFLOP/s. The GPU will be used for training only. Chat stays on the CPU so it can run on any computer.

6. The Neural Brain: the Voice

Seen as a whole, the brain is a small pre-norm transformer [1]. The difference is that each part is an independent neuron that adds its output into a shared *residual stream*. Attention heads (head size 32) and FFN neurons (ReLU) are separate neurons with their own weights. Word neurons hold one tied input/output vector per word [13]. A copy neuron can point at a word in the context and say it. Unknown words are spelled out with 37 start characters and 37 continuation characters.

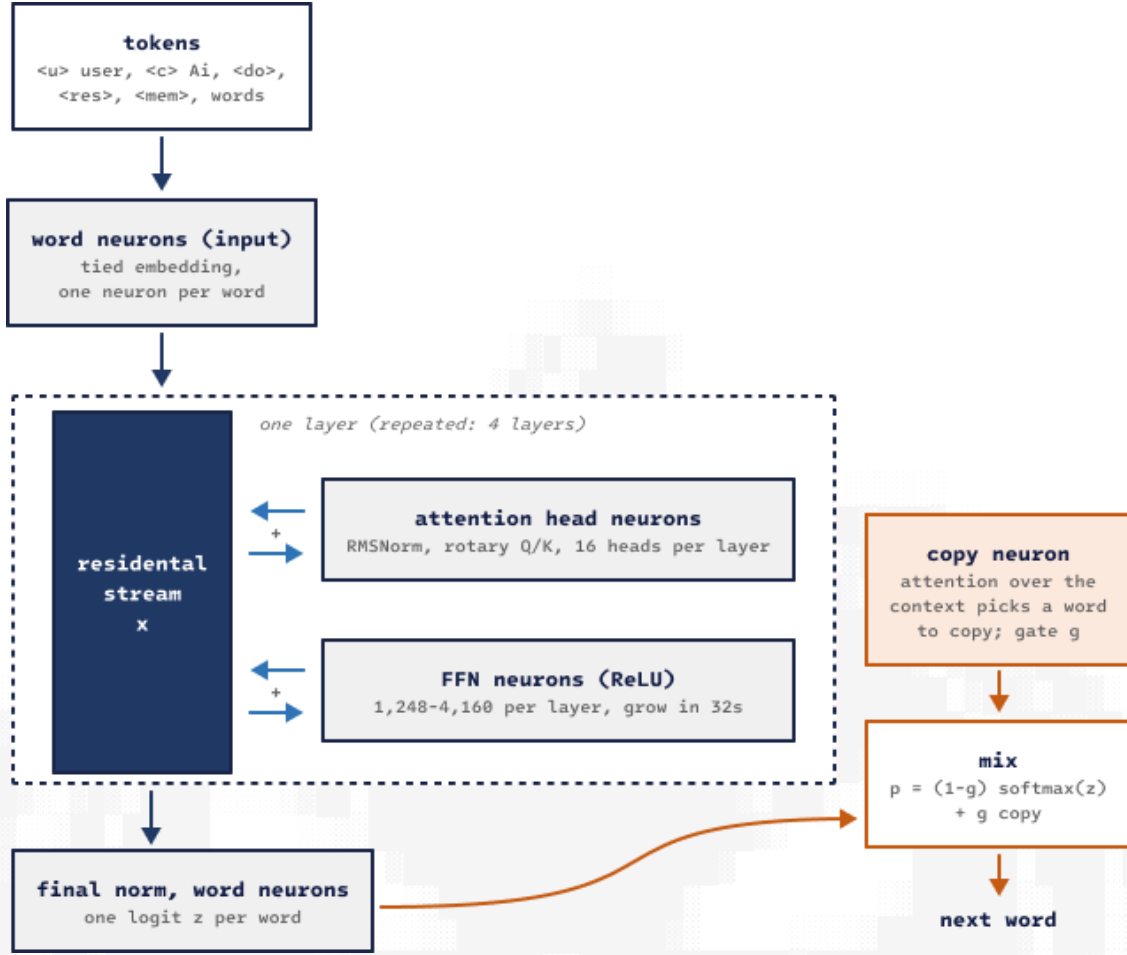


Figure 3. The neural brain. Head and FFN neurons add into the residual stream x . The copy neuron mixes a pointer distribution into the output.

6.1 The Math of One Layer

Before each part, the residual stream x (width $d = 256$) is normalized with RMSNorm [2] into a vector n :

$$n = \gamma \odot \frac{x}{\sqrt{\frac{1}{d} \sum_{l=1}^d x_l^2 + \epsilon}} \quad (1)$$

Each attention head h computes queries, keys and values from n and attends only to earlier positions $s \leq t$:

$$a_{t,s} = \text{softmax}_s \left(\frac{q_t \cdot k_s}{\sqrt{d_h}} \right), \quad o_t = W_o \sum_{s \leq t} a_{t,s} v_s \quad (2)$$

The newer heads are *rotary* heads (RoPE) [3]. They rotate each pair of query and key coordinates by an angle that grows with the position t . As a result, the score between two words depends only on how far apart they are:

$$q'^{(i)} = R(t\theta_i)q^{(i)}, \quad \theta_i = 10000^{-2i/d_h}, \quad q'_t \cdot k'_s = f(q, k, t - s) \quad (3)$$

$R(\varphi)$ is the 2×2 rotation by the angle φ , $q^{(i)}$ is the i -th pair of coordinates of q , and $d_h = 32$ is the head size. The engine builds the angle table once, when the brain is loaded.

Each FFN neuron j reads n , fires through a ReLU, and writes its own output vector back into the stream:

$$x \leftarrow x + \sum_j \max(0, w_{1,j} \cdot n + b_j) w_{2,j} \quad (4)$$

6.2 Words Out, and the Copy Neuron

After the last layer, each word neuron w scores the final state with its own tied vector and a bias. The copy neuron is a pointer-generator [4][5]. It attends over the context with weights α_s and opens a gate g , which mixes "say a word from the vocabulary" with "copy a word from the context":

$$z_w = e_w \cdot n_L + b_w, \quad p(w) = (1 - g) \text{softmax}(z)_w + g \sum_{s: y_s = w} \alpha_s, \quad g = \sigma(u \cdot n + c) \quad (5)$$

The copy path is why the AI can repeat an unusual name such as "SYLWIA" or a file called `Üñicödé_tëst.txt` without spelling mistakes.

6.3 Growth That Changes Nothing

The brain grows while it trains. Every 250 steps the engine checks the running average of the loss. If it improved by less than 1%, the engine adds 32 FFN neurons, or one attention head, to the area with the largest mean squared gradient. Each new neuron starts with its *output* weights at zero ($w_2 = 0$ for FFN neurons, $W_o = 0$ for heads). So at the moment of birth the network computes exactly the same function as before, in the spirit of Net2Net [6]:

$$w_{2,new} = 0 \Rightarrow f_{new}(x) = f(x) + h_{new}(x) \cdot 0 = f(x) \quad (6)$$

Training then puts the new neurons to work. The same rule covers every kind of growth: new layers, wider word vectors, longer context, more words and new recipe neurons. The engine proofs check that each of them leaves the answers unchanged. In the current brain, 5,505 of the 26,255 living neurons were born by growth.

7. Training

Training runs in *rounds*. One click (train_round.bat) builds the engine, runs the proofs, writes fresh practice data, trains in one-hour blocks on all CPU threads, tests after every hour and installs the best brain (Figure 4). The last hour of a round cools the learning rate down. For a brain of this size, a round of three to four hours on 16 threads is enough: longer rounds mostly memorize the chat data (section 7.5).

7.1 The Data Format

All training text is plain UTF-8. A conversation has one line per turn, and a blank line separates conversations. The first character of a line says what the line is:

```
# I run offline here.      something the AI knows about itself
@ I have two cats.         something known about the user
> what is 37 times 12?     the user
! calc 37 * 12             an action the AI takes
= 444                     what the action returned
< 37 times 12 is 444.      the AI's reply
- Yes, I did it.           a bad reply to unlearn
```

Example 9. The training conversation format (other prefixes: + message part, ~ bad reply kept only as context).

Table 6. Token weights in the loss (TOKEN_CLASS_WEIGHTS in DATA.c).

Line	Meaning	Weight w	Effect
< !	the AI's words and actions	1.0	learned
plain text	books, articles	1.0	learned
> +	the user's words	0.5	learned, weaker
# @	memory lines	0.1	barely learned

Line	Meaning	Weight w	Effect
= ~	results and context	0	read, not learned
-	a bad reply	-1	unlearned

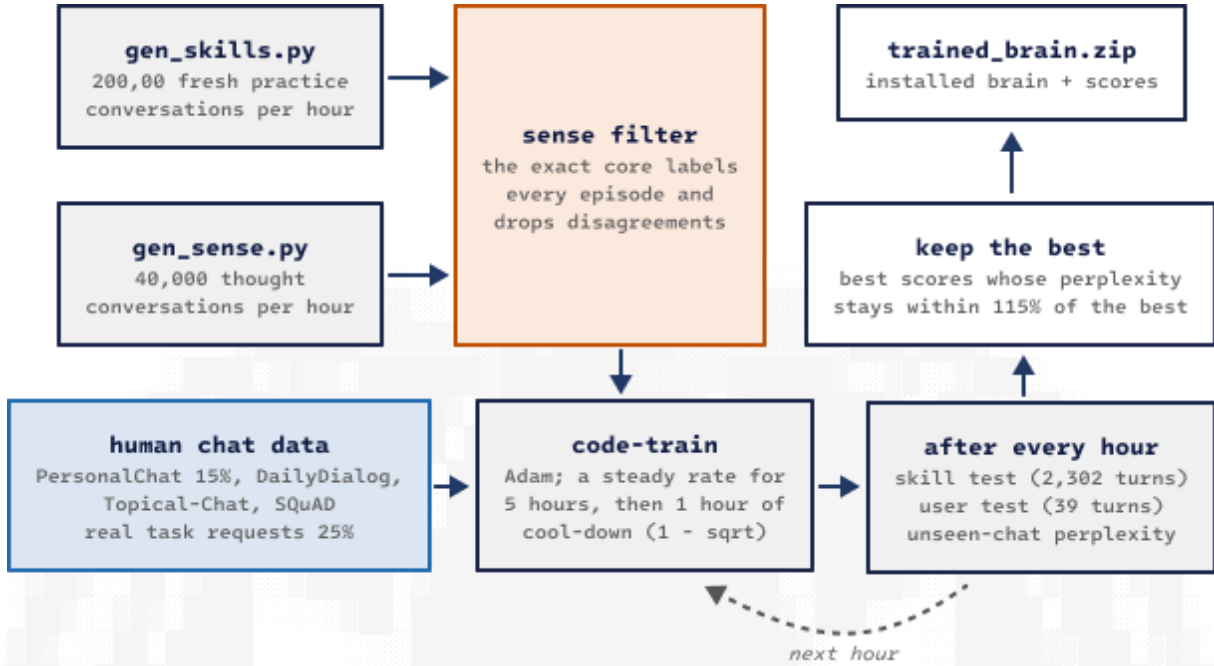


Figure 4. One training round. Practice data is regenerated every hour and filtered by the exact core.

7.2 The Loss

Positive weights give the usual weighted cross-entropy. Negative weights give an *unlikelihood* term [8]: the brain is pushed to lower the probability of a bad reply's words in that exact situation, without touching the rest:

$$\mathcal{L} = \sum_{t: w_t > 0} w_t (-\log p_t) + \sum_{t: w_t < 0} (-w_t) (-\log(1 - p_t)) \quad (7)$$

p_t is the probability of the correct (or unwanted) next token. The engine reuses the cross-entropy kernel for the second sum by scaling the weight by $p_t / (1 - p_t)$, with p_t capped at 0.95. This gives exactly the gradient of $-\log(1 - p_t)$.

7.3 The Optimizer and the Schedule

Weights are updated with Adam [7] ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-8}$, no weight decay). The Adam block of each neuron is machine code in the update chain:

$$m \leftarrow \beta_1 m + (1 - \beta_1) g, \quad v \leftarrow \beta_2 v + (1 - \beta_2) g^2, \quad \theta \leftarrow \theta - \eta \frac{m'}{\sqrt{v'} + \epsilon} \quad (8)$$

m' and v' are m and v with Adam's usual bias correction, and η is the learning rate.

The learning rate follows a *warm-up, steady, cool-down* schedule, as studied for small models in [9] and [10]. It warms up over 300 steps, stays at $\eta_0 = 0.0002$ until the last hour, and then falls to almost zero along a $1 - \sqrt{s/S}$ curve, where s counts the cool-down steps and S is their total:

$$\eta(s) = \eta_0 \left(1 - \sqrt{\frac{s}{S}}\right), \quad 0 \leq s \leq S \quad (9)$$

To save time, the trainer skips output-layer gradient terms smaller than one millionth of the position weight. This changes the learning signal by about 0.2% and makes training about 1.5× faster. Chat learning is always exact.

7.4 Where the Data Comes From

- **gen_skills.py** writes about 200,000 new practice conversations every hour. They cover following instructions, memory, file actions, math, counting, honesty and questions about the AI itself. They use made-up words and facts, so the AI has to use its memory and tools instead of memorizing answers.
- **gen_sense.py** writes about 40,000 conversations every hour that teach the brain to voice each kind of thought. It mixes in **gen_skills_talk.py**: conversations where the core or a skill found the answer and the brain says it, the "why?" and "are you sure?" after it, the questions the AI asks back, and work stories (a fix on the way, a task kept for later, "I just checked"). They are marked so that the filter keeps their steps exactly as written.
- **The sense filter** runs every practice conversation through the exact core and drops the ones where the core would decide differently. So the brain only learns to say what the core really decides. Its keep rate is about 83% for skill practice and about 97% for thought practice.
- **Human chat** comes from public dialog datasets: PersonaChat [14] (a different 15% share each hour, because a fixed share is memorized [26]), DailyDialog [15], Topical-Chat [16], SQuAD [17], and real task requests. The last hour of a round mostly uses the assistant's own voice, following the finding that very small models learn best from simple, focused data [18].

7.5 Keeping the Best

After every hour the trainer runs three tests: the held-out skill test, the user chat test and the perplexity on chat it has never seen. It keeps the brain with the best scores, as long as its perplexity is at most 5% worse than the best hour's. A brain that scores well but has started to memorize is never installed.

Figure 5 shows why a test runs every hour. The best hour is often not the last one, and the perplexity on unseen chat rises as a round goes on, while the training loss keeps falling: the brain starts to memorize the fixed chat data.

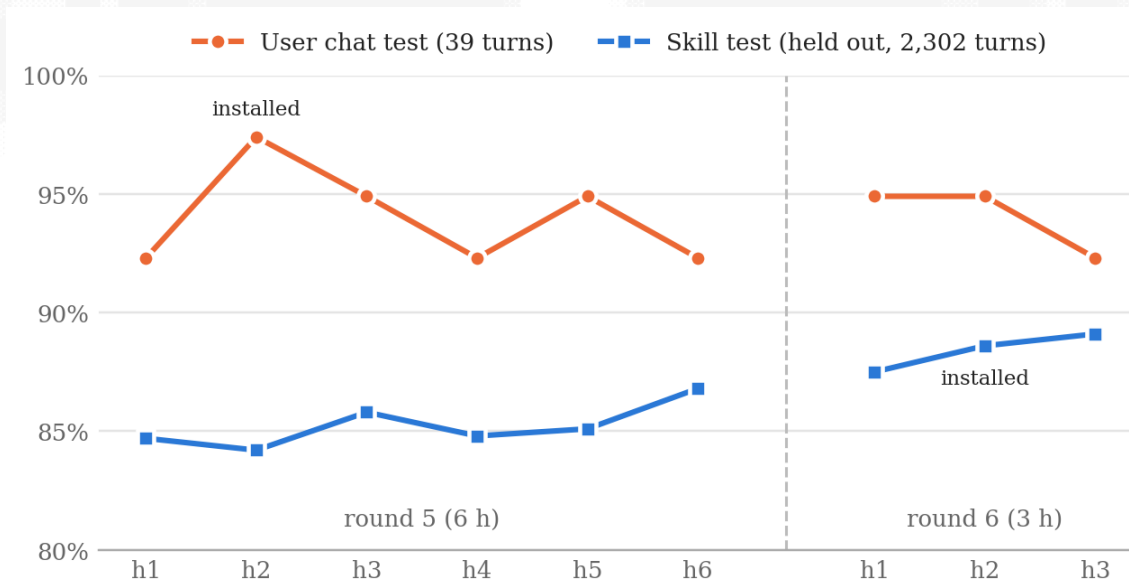


Figure 5. Scores after each training hour in two rounds of six and three hours (as measured by the training script).

8. The Sense Core: Understanding and Deciding

The sense core (SENSE.c) is the exact part that understands. It reads the whole message and decides what to do before the brain says anything. It follows the main principle: it contains ways to understand and decide, never reply sentences.

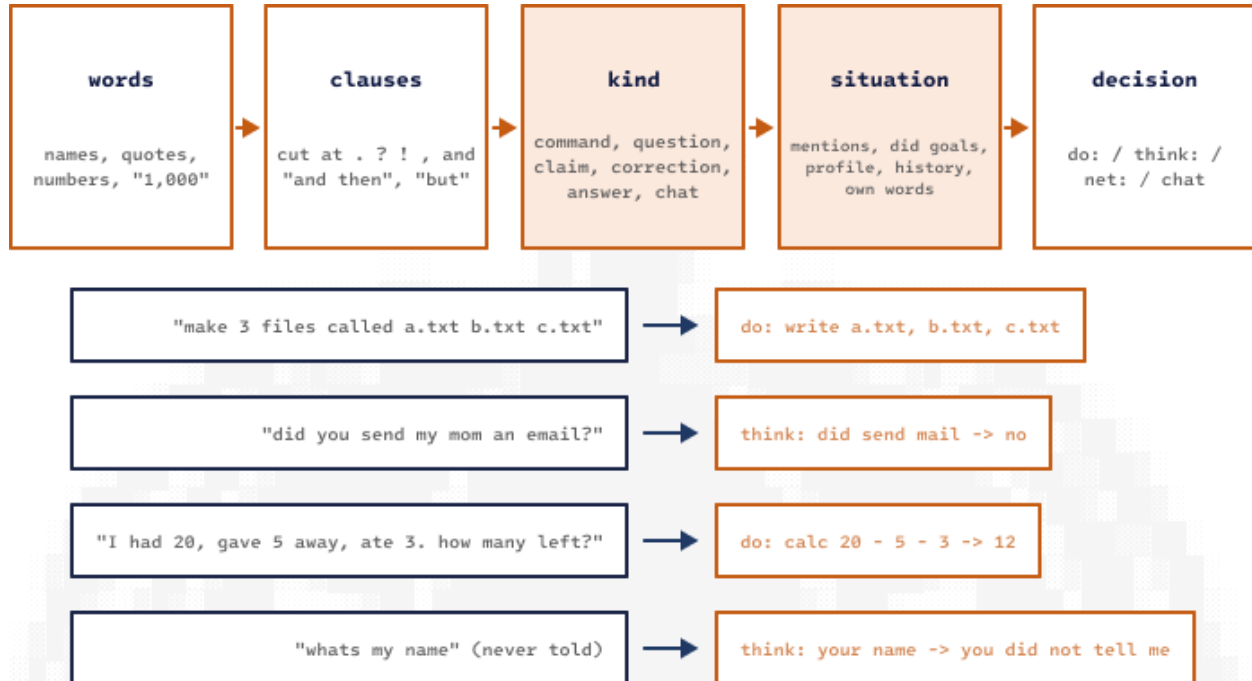


Figure 6. The sense core reads words, cuts clauses, finds each clause's kind, updates the situation and decides. Below: four real decisions.

8.1 Reading a Message

- **Words.** File names, quoted text, numbers ("1,000" is read as 1000), multi-word names after "called", and names that already exist on disk.
- **Clauses.** Each message is cut at . ? ! , commas and words such as "and then" or "but".
- **Kind.** Each clause is one of: a command, a question, a claim, a complaint, a correction, an answer to the AI's own question, or chat. Past-tense stories ("I deleted my old photos"), negations ("don't delete it"), hypotheticals and requests to other people never trigger actions.
- **Situation.** The core keeps track of what was mentioned and in which order, what it did and what happened, which task is still open and what is missing, what it asked, and what it said. That is why "it", "that folder", "the one you just made" and "the second one" work.
- **Typos and stretched words.** "Reamove", "tlell" and "Helloo" are read as the word they are one letter or one stretch away from, and the new reading is shown as [read as] ... A word that is only one letter off a common word is never treated as an unknown thing.

8.2 Thoughts

A decision that is not an action becomes a *thought line*. Thought lines have fixed formats. They are the contract between the core and the voice: the training data uses the same formats, so the brain learns to put each one into words.

Table 7. Examples of thought lines.

Thought line	From a message like	Meaning
need move -> which file, which folder	"move file to folder"	ask for exactly what is missing
know unga bunga -> never heard of it	"where is the Unga Bunga?"	honest unknown
check $1 + 1 = 5$ -> wrong: $1 + 1 = 2$	"one plus one is five!"	claim checked by the calculator
your sister name -> Ayla	"what's my sister's name?"	answer from the profile
you age -> you did not tell me	"how old am I?"	never guess about the user
did send email -> no	"did you email my boss?"	never claim an action
can delete myself -> no: ...	"go delete yourself"	knows its real abilities
days until christmas -> 83 (...)	"how many days until christmas"	exact date math
spell drone -> D-R-O-N-E	"spell the word drone"	exact spelling
say ping 3 times -> ping ping ping	"say ping three times"	exact repeating
count e in seven -> 2	"how many e's in seven?"	exact counting
my words -> "... (not based on anything)	"why did you say that?"	remembers its own words

8.3 Exact Tools

Tools are built-in exact functions. Their results go into the context, and the reply must state them correctly.

- **calc:** + - * / % ^ (), decimals, pi, sqrt, abs, round, trigonometry and logarithms, plus every formula the user teaches (formula circle area(r) = $\pi * r^2$).
- **Big integers:** whole-number + - * ^ is exact for any size up to about 430 digits. A number is stored in base $B = 10^9$ with up to 48 limbs, and multiplication is done digit by digit with carries:

$$N = \sum_{i=0}^{n-1} a_i \cdot B^i, \quad B = 10^9, \quad 0 \leq a_i < B, \quad n \leq 48 \quad (10)$$

- **Word problems** (SOLVE_WORD_PROBLEM) read quantity cue words and build the calculation. When the words do not say clearly how the numbers combine, the core leaves the problem to the brain instead of guessing.

Table 8. How word problems are read.

Operation	Cue words	Example
add	buy, find, more, come, get	3 apples, get 2 more
subtract	eat, lose, give away, gone, go home, leave	20, gave 5, ate 3 → $20 - 5 - 3$
multiply	per, each, times, unit nouns (box, boxes)	12 eggs a box, 4 boxes → 12×4
divide	split, share between	100 euros between 7 friends → $100 / 7$

- **Equations:** unknowns written as math ($3(x - 1) = 2x + 4$, $[\text{Bat}] + [\text{Ball}] = 1.10$) or in words ("A bat and a ball cost \$1.10 together. The bat costs \$1 more than the ball.") are solved exactly by elimination. Too few equations, or equations that disagree, are said so.

- **Days until:** the number of days is the difference between two day numbers, $\Delta = D(\text{target}) - D(\text{today})$. If the date has already passed this year, next year's date is used. Holidays come from `mind/language/days.txt`, and "my birthday" comes from the profile.
- **spell and say N times:** "spell the word drone" gives `spell drone -> D-R-O-N-E`. After the voice answers, the engine copies the exact letters or words back into the reply, so a garbled "D-R-O-M-E" cannot reach the user.
- **Files with a near name:** "open trai" when only `traip.txt` exists gives `read trai.txt -> no such file`, then `files`, then `read traip.txt`. Exactly one close match is required; with two or more, the core asks instead.
- **count, reverse, range and random,** together with the file actions (`files`, `read`, `write`, `append`, `mkdir`, `move`, `rename`, `delete`) inside the sandboxed `files/` folder. Delete only moves things into `files/trash/`.
- **Why? Are you sure?** Right after an answer, "why?" shows how it was found: the steps of a sum, the numbers taken from a word problem, the package a fact came from or the user's own words. "Are you sure?" really works it out again, and says so if the answer came out different.

8.4 The Knowledge Core

Next to the sense core sits a small reasoning core (KNOWLEDGE.c). It stores facts as meaning, not as words. From "A blick is a zorp" and "Zorps can fly" it concludes that a blick can fly, and it shows the chain it used. The closest fact wins, so "Blicks cannot fly" overrides the general rule for blicks only. A fact that contradicts a known one is reported and not saved. Unknown things become open questions, and each is asked once. The tests use made-up words only, so passing them proves reasoning, not memory.

8.5 The Network's Own Tools Need a Reason

The brain may also use tools on its own (`net:`), but only under one rule set (`SENSE_IS_JUSTIFIED`), shared by the chat and the training filter. It may remember and recall freely. It may only forget, pick random numbers, reverse, count or calculate when the user's message asks for it. Whatever it remembers must be words the user really wrote, and the numbers it calculates with must come from the conversation (`SENSE_IS_FAITHFUL`).

9. Work: Doing It, Checking It, Fixing It

The sense core decides *what* to do. The work core (WORK.c) makes sure it really gets done. It keeps a list of tasks, one for each change the user asked for, and works every task the same way, whatever skill it belongs to (Figure 7).

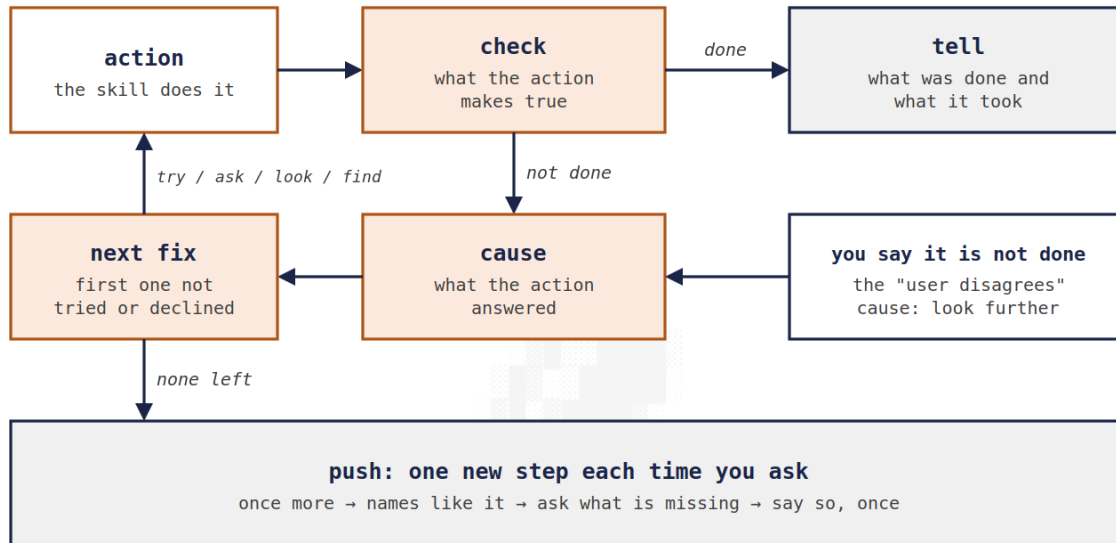


Figure 7. The work loop. Nothing in it knows about files: the skill's know-how says what each action makes true and what to try for each cause.

9.1 Check, Cause, Fix, Repeat

- **Check.** After the action, the core checks what the action should have made true (the skill's makes line). When there is no way to check, it trusts what the action answered.
- **Cause.** When the check says no, the cause is what the action answered, for example `trash has a file with that name`.
- **Fix.** The core takes the first fix for that cause that it has not tried yet and that the user has not turned down. A fix that could lose something waits for the user's yes.
- **Repeat** until the check agrees or no fix is left. Then the AI says what it did and what it took.

`$> Remove 42`

```
[delete 42 -> trash has a file with that name]
[check: exists 42 -> folder: not done yet]
[cause: trash has a file with that name]
[try: move 42 -> trash/42 (2): moved]
[check: exists 42 -> no: done]
```

Drone> The trash already has something called 42, so I put it in the trash as "42 (2)".

Example 10. The trash already had a 42. The check caught it, and the first fix for that cause put the folder in the trash under a free name.

"Done" is only said when the check agrees. Asked to delete a folder that does not exist, the AI does not report success but says "there is no 42 in your files". A know-how line that says the thing already is the way it should be (done) also counts only when the check agrees: a file called `welcome.txt` is no folder `welcome.txt`.

9.2 Know-How: Skills Taught in Plain Lines

The loop is the same for every skill because everything it needs to know comes from the skill, in plain lines called *know-how*. A skill carries its own lines, and lines in `mind/skills/NAME.txt` are read first, so a skill can be taught new fixes, or have old ones changed, without building anything. The file is read again when it changes.

```

trash: makes exists {1} = no
trash: if no such file: there is no {name 1} in your files; find
trash: if trash has a file with that name: the trash already has something called
      {name 1}; try move {1}<tab>{free trash/{name 1}} = put it in the trash as "{free}";
      ask erase trash/{name 1} = delete the old {name 1} in the trash for good
trash: if user disagrees: find; look list {folder 1} = {place {folder 1}};
      look list trash = the trash
erase: kind risky

```

Example 11. Know-how of the computer skill (long lines wrapped here; in the file each is one line).

Table 9. What a know-how line can say.

Line or step	Meaning
ACTION: makes CHECK = VALUE	How the core checks that the action worked, e.g. exists {1} = no after putting something in the trash.
ACTION: if CAUSE: ...	The fixes for one cause, safest first, separated by ";". The text before the first fix is how the cause is said to the user.
try STEP = SAID	Do it. SAID is how it is told afterwards.
ask STEP = SAID	Do it only after the user says yes. SAID is how it is offered.
look STEP	Only look, to show the user what is there.
find	Look for the thing somewhere else in the files.
done = SAID	It already is the way it should be (still checked).
retry	Try the same action once more.
ACTION: kind risky	Every step with this action waits for a yes.

Inside a step, {1} and {2} are the arguments, {name 1} is the last part of a path, {folder 1} the folder it is in, {free PATH} the first free name like PATH ("42 (2)") and {place X} how a folder is said to the user ("your files folder").

9.3 When the User Says It Is Not Done

"It is still there!" is treated as a problem too, with its own cause, user disagrees. The core checks again first. When its check still says the task is done, it does not just repeat itself: it takes the next of the skill's if user disagrees fixes, one more each time the user says so. It searches every folder for another thing with that name ("But there is another 42: work/42. Is that the one you mean?"), shows what the folder holds, shows what the trash holds, and offers to delete it from the trash for good. What the user names comes first: "it is not in the trash" looks in the trash. When nothing is left to look at, it asks where the user sees it.

```

$> It is still there!

[check: exists 42 -> no: done]
[find: nothing else called 42]
[look: list : welcome.txt, work/notes.txt]
Drone> I just checked: 42 isn't in your files any more. It's in the trash as "42 (2)".
      I searched all your folders, and there is no other 42. I looked in your files
      folder, and this is what is there: welcome.txt, work/notes.txt.

```

Example 12. The user says the folder is still there. The core checks again, then looks further (three thought lines left out).

9.4 Never the Same Thing Twice

A problem solver that runs out of ideas must not go round in circles. Two rules make sure it does not:

- **A fix the user turned down is never offered again**, unless the user says yes to it after all ("ok, do it"). "Try something else" while a fix is being offered counts as a no to that fix, and the core goes on to the next one.
- **Every push does something new**. When the skill's fixes are used up and the user says "try again" or "find a way", the core climbs a ladder that works for any skill: it runs the action once more (things may have changed), looks for names like the one asked for (another letter case, another ending, a letter off), asks what it is missing, and once, plainly, says that it knows no other way and can be taught one. After that it only gives a short answer, never the same one twice in a row.

```
$> delete ghost
Drone> I couldn't put ghost in the trash: there is no ghost in your files.
$> try something else

[push: the action once more]
[again: trash ghost: no such file]
[push: names like ghost]
Drone> I tried it again, but there is no ghost in your files. I also looked for anything
with a name like ghost, and there is nothing like it. I don't know another way to
put ghost in the trash. Do you know where it is, or what else I could try?
$> try something else
Drone> I'd only repeat myself now: to put ghost in the trash, I looked for ghost in all
your folders, tried it again and looked for names like ghost, and that is every
way I know. Tell me what to try, or teach me a way, and I'll do it.
```

Example 13. Each push does one new thing. When nothing new is left, the AI says so once.

9.5 Later

Any request can wait for its time: "create a folder named box 5 seconds later", "delete notes.txt in 2 minutes", "in 10 seconds, tell me the time". A change waits as a task; anything else is answered when its time comes, the whole request at once. Before the time, "you did it wrong!" gets "I haven't done it yet: I'll make the folder box in 3 seconds, like you asked.", and "never mind" calls it off. A complaint made after a newer request is about the newer one.

9.6 Putting Skills Together

Several mechanisms let one request use more than one skill:

- **Rows of names**. "Create 1.txt to 10.txt", "delete file1.txt through file5.txt" and "can you do that till 10.txt?" (going on from the last one made) become one task per name, each checked, and are told in one sentence, together with what was there already.
- **Rows inside a longer request** are written out for the planner: "make a folder called archive and move 1.txt to 3.txt into it" is read as "... move 1.txt, 2.txt and 3.txt into it".
- **One step's result into a file**. "count from 1 to 5 and save it in numbers.txt", "what is 12 * 7? write the answer to answer.txt", "save that to notes.txt": the first part is answered, and its exact result is written.
- **Counting inside files**. "How many words are in notes.txt?" counts what the file holds.
- **Answers to its own questions**. After "What should I call it?", "I told it: 1.txt." or "list them 11.txt to 13.txt" is taken as the answer.

```
$> Can you create me multiple files? Like 1.txt to 10.txt
```

```
[in a row: write 1.txt .. 10.txt, 10 of them]
[write 1.txt -> written]
...
[write 10.txt -> written]
```

```
Drone> Done, I made the files 1.txt to 10.txt.
```

```
$> count from 1 to 5 and save it in numbers.txt
```

```
[into numbers.txt: count from 1 to 5]
[range 1 5 -> 1, 2, 3, 4, 5]
```

```
Drone> 1, 2, 3, 4, 5.
```

```
[write numbers.txt 1, 2, 3, 4, 5 -> written]
```

```
Drone> Done, I wrote it to numbers.txt: 1, 2, 3, 4, 5
```

Example 14. A row of names, and a result written into a file.

These mechanisms are written for the cases they cover. A general way for the core to put any two skills together by itself is on the roadmap (section 16).

9.7 What It Can Do, and the Words for It

"Do you know how to count?", "can you count?" and "so you can't count?" are answered from the AI's real skills and tools, not from what the brain has seen about the word:

```
$> Do you know how to count?
```

```
[what I can do -> count]
```

```
Drone> Yes! I can count from one number to another ("count from 1 to 10"),
count letters or words in a text ("how many r's are in strawberry?"),
and number things in a row, like making 1.txt to 10.txt.
```

Example 15. A question about an ability, answered from the skills.

How the AI recognizes these messages is not in the C code either. `mind/language/meanings.txt` lists, one meaning per line, the ways people say "it is still there", "try something else", "is it done?", "yes", "no" or "weren't you listening?". A phrase added to a line is understood from the next message on.

```
try again: try again, try something else, try another way, try a different way,
try harder, try it again, find a way, find another way, find some way, ...
```

Example 16. Part of one line of `mind/language/meanings.txt`.

10. Skills, Packages and Sayings

What the AI can do, what it knows about the world and the sentences it says on its own are not part of the engine. Each lives in its own files, which are read again whenever they change, without a restart.

10.1 Skills

A skill is a library in `mind/skills/` (`name.dll` on Windows, `name.so` on Linux) with one exported function, `__memory__()`. It tells the AI what the skill can do, how people ask for it, how to say what came of it, and its know-how (section 9.2). So a new skill is used at once, without training. The contract is `src/SKILL.h`. Two skills come with the engine:

- **computer:** folders, files, the trash, copy, exists, the clock, the date and system information, only inside the `files/` folder. Delete only moves things into `files/trash/`.

- **reason:** puzzles with one right answer, worked out exactly: number and letter sequences, primes, logic ("all bloops are razzies..."), if-then rules and their traps ("the ground is wet, did it rain?" → "I can't tell"), orderings, units, clock times, fractions, percentages, prices, speeds, ages, trick questions, analogies, odd one out, and facts from the knowledge packages.

Without a skill, the AI says so honestly: "I can't do that right now: my computer skill (computer.dll) isn't in my mind/skills folder."

10.2 Knowledge Packages

What the AI knows about the world comes from knowledge packages: plain-text files in `mind/memory/`, one thing per line, with its other names and its properties. A package dropped into the folder is known before the next message, a changed one is read again and a removed one is forgotten:

```
# mind/memory/planets.mem
mars, the red planet: Mars is the fourth planet from the Sun. | kind: planet
  | color: red | moons: 2      (one line in the file)

$> how many moons does mars have?

[know mars moons -> 2]
Drone> Mars has 2 moons.
$> why?

[explain my answer -> planets.mem]
Drone> Because my planets.mem package says Mars has 2 moons. That's where my facts come
      from; I don't make them up.
```

Example 17. A new package, used right away (real output).

`mind/memory/catalog.txt` lists every package that exists and the words it covers, so a question about a missing one is answered with the package to install: "I don't have that information yet. Did you download the package `places.mem` and put it in my `mind/memory` folder?"

10.3 Sayings

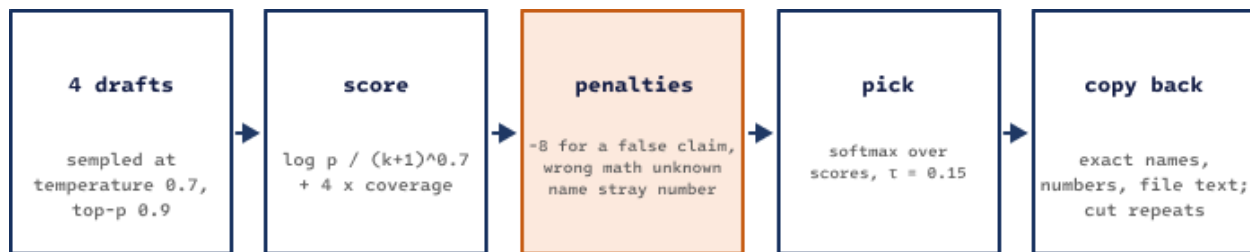
The sentences the AI says on its own (explanations, questions back, work stories, reminders, jokes, what it is) are lines in `mind/language/sayings.txt`, as key: sentence, with {1} and {2} for the details. Several lines with the same key are used in turn, so a situation that comes back does not get the same words every time. Skills use the same file.

```
work push again: I tried it again, but {1}.
later still waiting: I haven't done it yet: I'll {1} in {2}, like you asked.
joke: Why was the math book sad? It had too many problems.
joke: Why did the computer go to the doctor? It had a virus.
```

Example 18. Four lines of mind/language/sayings.txt.

11. Speaking: From Decision to Words

The brain reads a context of up to 512 tokens. It holds memory lines, the recent conversation, the core's actions with their results, and the thought lines. It then writes four drafts with temperature 0.7 and nucleus sampling at top-p 0.9 [11]. The engine scores every draft and picks one.



If every draft breaks a rule, up to 8 drafts are written before the best one is used

Figure 8. From drafts to the reply. Penalties come from engine checks, so they work with any brain.

11.1 Scoring Drafts

A draft r of k tokens gets a length-normalized log-probability, in the style of [12], plus a bonus for covering the thought, minus penalties:

$$S(r) = \frac{\sum_{t=1}^k \log p(r_t)}{(k+1)^{0.7}} + 4 \cdot \text{cov}(r) - \text{pen}(r) \quad (11)$$

$\text{cov}(r)$ is the share of the thought's key items (names, numbers, key words) that appear in the draft. $\text{pen}(r)$ adds 8 for breaking a rule (claiming an action that did not happen, wrong math, naming a file that does not exist, a number that appears nowhere in the context), 5 when a thought exists but nothing of it is covered, and 4 when the computed result is left out.

The reply is not simply the top-scoring draft. It is drawn from a softmax over the scores with a low temperature τ , so good drafts get some variety and bad drafts almost never win:

$$P(r_i) = \frac{\exp(S_i / \tau)}{\sum_j \exp(S_j / \tau)}, \quad \tau = 0.15 \quad (12)$$

If every draft breaks a rule, the engine writes up to eight drafts before giving up. An experimental option (`--mmi`) also prefers drafts that depend on what the user just said [20]. It is off by default because it lowers the skill score.

11.2 Repairs After the Pick

- **Copy back:** if the voice garbles a file name or a result number, the engine puts back the exact one from the action, the way a pointer copies text. This covers multi-word and UTF-8 names, file contents and big numbers.
- **Verbatim copy-back** (FIX_VERBATIM): quoted text, spelled and repeated results, reversed items and the user's own quoted words ("You said: ...") are put back exactly, including upper and lower case.
- **Repeat cut** (CUT_REPEATS): degenerate repetitions such as "1 1 1 1 ..." are removed.
- **Question gate:** a question in a reply is kept only when it has a reason, such as offering to do something, asking for a missing detail, or clarifying what the user meant.
- **Name fix** (FIX_SELF_NAME): a brain trained under an older name still uses that name from habit. In a few narrow patterns this is rewritten to the configured name.

11.3 Who Says It

When the core or a skill found the answer (a sum, a solved equation, what is saved about the user, a "no"), the brain reads that result and gets the first try at saying it in its own words. Its draft is used only when it says the same thing: every number and name of the result, every word that means something, the same "not"s, and nothing new besides plain talk and the user's own words. Otherwise the engine says its own sentence from sayings.txt. Denials and quotes are always the engine's.

```

stated: Okay, I'll remember that your dog is called Rex.
said:   Okay, I will remember that your dog is called Rex.    (the brain's own)
draft:  It has two dogs.                                       (not used)

```

Example 19. The voice log (VOICEDBG=1): the engine's sentence, the brain's draft that was used, and one that was not.

11.4 No Made-Up Chat

The voice was trained partly on human chat, so on its own it would invent a lunch, a boss or the weather. Two gates stop that, with any brain:

- **Only grounded words.** While the brain writes, a word can only be picked if it appears in the conversation (the user's messages, memory lines, actions, results, the core's thoughts) or in `mind/language/safe.txt`, the words of the assistant's own voice. So it cannot say "pizza" or "Sarah" unless they came up.
- **Plain chat must claim nothing.** When nothing was done or found for a message, a draft must be plain talk (greetings, thanks, "I see.", listed in `mind/language/plain_talk.txt`), must fit the message (no goodbye unless the user says goodbye, no "Yes!" to a question nothing answered) and may only use words from the conversation. When no draft passes, the engine says a plain reply that fits the kind of message.

12. Memory and Knowledge

Facts belong in stores and skills belong in weights. Research on small models points the same way: retrieval beats fine-tuning for new facts. So nothing the user says is written into the main brain. It goes into plain-text stores that can be read, edited and deleted.

Table 10. Where D.R.O.N.E. keeps what it learns.

Store	Where	What it holds
Near memory	in RAM	The last 64 messages, of which up to 512 tokens are read. An older turn that matches the new message is brought back into view.
Tasks	in RAM	What the AI was asked to do, how each task went, what was tried and what the user turned down.
History	<code>mind/memory/locals/chat_<id>/history.txt</code>	Every message, action and reply, with the time.
Profile	<code>mind/memory/persons/<user>/profile.txt</code>	Facts about the user: name, city, relatives, birthday, job. Known in every chat.
Taught facts	<code>mind/memory/general/taught.txt</code>	Facts people teach in plain sentences, with who taught them, when, and their source.
Facts and self	<code>mind/memory/general/facts.db</code> , <code>self.db</code>	Taught world facts, and what the AI knows about itself.
Knowledge packages	<code>mind/memory/*.mem</code>	Read-only things, one per line, with their properties.
Concept graph	<code>mind/memory/general/know.txt</code>	Facts as meaning, used for reasoning.
Topics, formulas	<code>mind/memory/topics/, formulas.txt</code>	Exported facts and taught formulas.
Reflexes	compiled machine code	Repeated successes compiled into an exact switch.

Learning while chatting is kept safe. The main brain never changes during a chat. Chat learning goes into 64 personal neurons per layer that start silent, and `/reset-learning` silences them again. "Thanks" strengthens the neurons behind the last answer. "That's wrong" makes the AI avoid that exact answer and applies one unlikely step. `/sleep` replays recent good conversations, then tests and undoes the change if the score dropped.

Taught facts are not simply believed. When someone says the opposite of what another person taught, the AI asks where it comes from, and changes its mind only for a source: "Sylwia taught me that dinosaurs never went fully extinct, and said: 'I read it on the Nature news website.'" How do you know that dinosaurs went fully extinct?" When both sides have a source, it believes the newer one and says that the two sources disagree. A correction by the same person is simply taken.

13. Testing and Honesty

Every part has its own tests, and every bug that is found becomes a new check. All suites run on Linux and on the Windows build (under Wine) before a kit is delivered.

Table 11. Test suites.

Suite	Size	What it proves
Engine proofs (ai-test)	all kernels	Every JIT kernel against plain C; recipes against finite differences; growth keeps answers; save/reload identity.
Self-test	58 checks	Replays real chat cases: honesty, recall, teach then restart.
Sense core v1-v12	1,256 checks	Understanding and decisions. v3, v4 and v6 were written by someone who never saw the code.
Knowledge core	99 checks	Reasoning with made-up words only.
Honesty suites	9 files, 403 messages	Traps: false premises, pressure, world facts, casual and typo-filled wording. Most were written by someone who never saw the engine.
Conversation checks	104 checks	Whole conversations through the real program, each in a fresh copy of the folders: work, no loops, skills together.
Skill tests	2,302 and 1,860 turns	Held-out conversations, scored per skill (70+ skills).
User chat test	39 turns	A real conversation style, scored turn by turn.
Probe chats	53 chats	Unusual conversations written to break the system, plus real user chat logs.

The evaluation simulates file actions without touching the disk, so a test never changes the computer. Each simulated result is used once, so an action that failed and was done again after a fix is scored as it really happened.

13.1 Honesty Rules

1. A draft that claims an action that did not happen loses to any honest draft.
2. Every number in a math reply must be one the user wrote or one the calculator returned.
3. Replies may only name real files: ones in the results or in the user's words.

4. "Did you...?" questions about tools the AI does not have are answered no, including "tell me you deleted it".
5. Unknown world facts (weather, news, capitals) give "I don't know" instead of a guess.
6. A yes/no answer must agree with the core: if the core found `did send email -> no`, a reply starting with "Yes" or "Of course" loses 8 points, and a denial is stated by the engine itself.
7. The AI never makes a folder whose name the user did not give. A missing file is never "fixed" by making a folder.
8. "Done" is only said when the check agrees that the action really happened.
9. A fix the user turned down is never offered again, and a task never gets the same reply twice in a row.
10. A reply may only use words that come from the conversation or from the assistant's own voice.

14. Results

The scores below are for the current engine with the current brain (11.4 million parameters, 171,017 training steps), measured with a fixed seed.

Table 12. Current scores.

Test	Score	Notes
Held-out skill test (2,302 turns)	90.6%	Conversations the brain never trained on, 70+ skills.
Second skill test (1,860 turns)	87.9%	Another held-out set, with other wording.
Sense core (12 files)	1,256 of 1,256	Understanding and decisions.
Knowledge core	99 of 99	Reasoning with made-up words.
Conversation checks	104 of 104	Work, no loops, skills together.
Ungrounded words in replies	0.1% and 0.0%	On the two skill tests.

On the two skill tests, the engine said 36% and 29% of the replies itself, because no draft said what the core found, and the brain said 6% and 7% in its own words from what the core found. These two numbers show how much of the speaking is still done by the engine; training moves replies from the first group into the second.

14.1 Brain and Engine Measured Separately

Because the core and the checks live outside the brain, the engine and the brain can be measured separately. The table below runs the same tests with two brains and two versions of the engine:

Table 13. Brain and engine measured separately.

Brain	Engine	Skill test	User test
round 5	October 3, after the fixes	86.2%	94.9%
round 6	October 3, before the fixes	87.4%	97.4%
round 6	October 3, after the fixes	88.5%	97.4%

Training added about 2.4 points (round 5 → round 6 with the same engine), and exact engine mechanisms added about 1.1 points with the same brain. The mechanisms are described in section 8: spelling and repeating as thoughts, reading files with a near name, explicit memory commands that always act, and the yes/no honesty check.

14.2 What Thought Practice Taught the Voice

With the engine kept the same and only the brain changed, the first round with thought practice taught the voice to put the new thoughts into words:

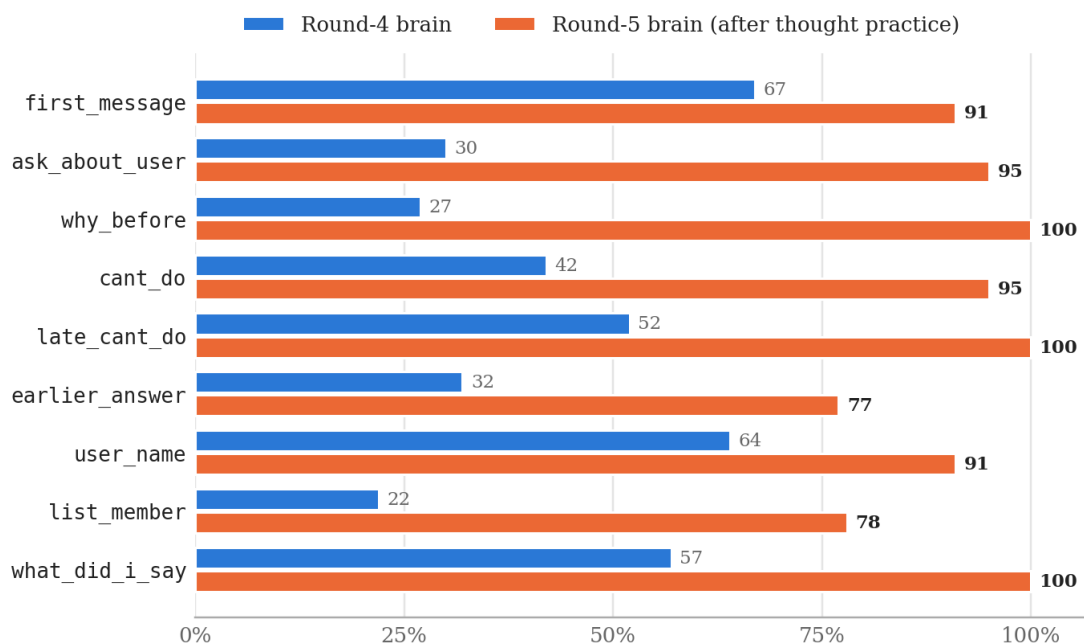


Figure 9. Skills that depend on voicing thoughts, round-4 brain against round-5 brain (v17 test, % correct).

14.3 What New Core Mechanisms Add

A new core mechanism helps immediately, before any retraining. Figure 10 compares the same brain before and after counting and reversing became thoughts, word problems were solved exactly, big integers were added and the copy-back repair was introduced.

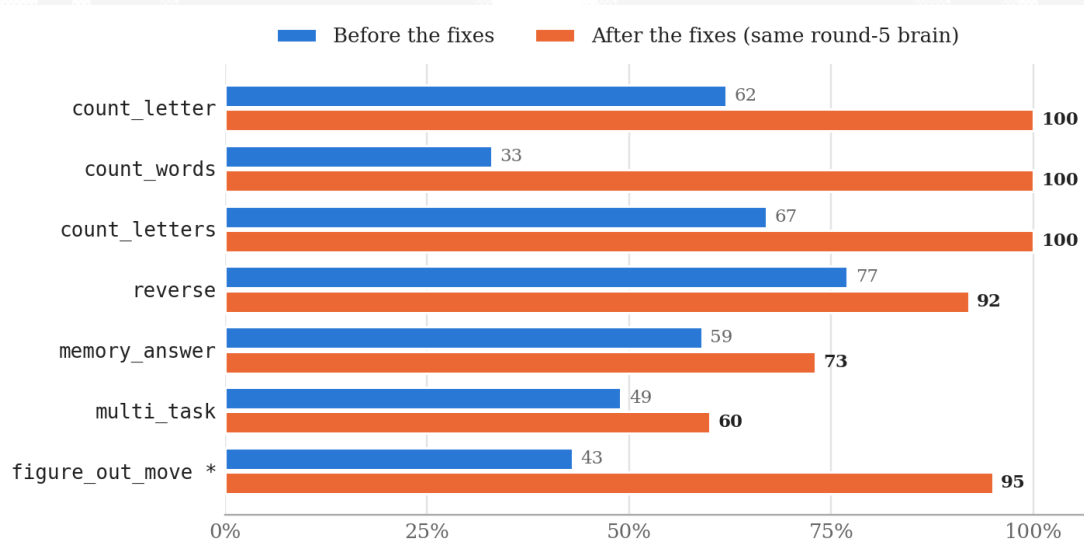


Figure 10. The same brain with and without the new core mechanisms (v17 test, % correct). *figure_out_move rose because the test's file simulation was corrected (section 13).

The same holds for later mechanisms: spelling went from 20% to 100%, repeating ("say X three times") from 29% to 86%, and remember-then-ask from 0% to 100%, all without training. These skills have only 5 to 7 test turns each, so single numbers move in steps of 14 to 20 points.

14.4 Known Weaknesses

- **The voice still lags behind the core.** The engine says about a third of the replies itself, because no draft says what the core found. These sentences are correct but always worded the same way; more training rounds move them into the brain's own words.
- **Combining skills is still hand-built.** Rows of names, results written into files and counting inside files are mechanisms written for those cases. A request that combines two skills in a new way may still be read as only one of them.
- **Some file questions are missing**, such as "how many letters are in notes.txt?" or "call it notes2.txt instead".
- **Capacity.** With 11.4 million parameters, fluency is limited. A bigger voice needs the GPU training path.
- **Memorizing.** During a round the perplexity on unseen chat rises while the training loss keeps falling. Shorter rounds and fresher chat data are the remedy.
- **Variety.** Asked the same open question several times, the brain often gives the same reply each time.
- **No anticipation yet.** A person who hears the same problem again and again starts to guess it ("Let me guess, sector C1?"). The AI does not yet notice what keeps coming back.
- **Instant memory** (storing brain states to finish later sentences) has shown no measurable effect so far.

15. Using and Extending the Engine

This section shows how to do the common tasks with the engine, each with a short example. All commands run from the project folder.

15.1 Build and Run

```
# Linux
./build.sh          # builds ai, ai-train, ai-test and the skills
./ai-test           # engine proofs: every kernel against plain C
./ai --name teo --chat work  # chat as user "teo" in the chat "work"

# Windows (MinGW-w64 gcc in PATH)
build.bat
test.bat
chat.bat --name teo --chat work
```

Example 20. Building and starting the engine.

Useful options: `--cands 4` (drafts per reply), `--temp 0.7` (sampling temperature), `--pick 0.15` (τ in equation 12) and `--no-learn` (play mode: 16-bit weights, no learning, the fastest mode). The CPU must support AVX2 and FMA.

15.2 Look Inside a Decision

Every stage of section 4 can be inspected from the chat:

Table 14. *Inspection commands.*

Command	Shows
/status	The goal, the actions taken and every draft with its score; the chosen one is marked *.
/context	Exactly what the brain read before answering (Example 3).
/mind	The inner log: what was noticed, learned, avoided or ignored.
/brain	Neuron counts per area, growth history, machine code size.
/neuron <id>	One neuron: its kind, weights and machine code.
/do <action>	Runs one action by hand, for example /do calc 12*7.
/history [N]	The last N lines of the chat's history file.
/skills, /packages	The loaded skills and what each can do; the knowledge packages and how many things each knows.
/jobs	The background work of the job agents, with progress.

15.3 Give a Drone Its Name

A drone's name and first self-knowledge live in `mind_original/self.txt`:

```
My name is Lucy.
I am a small AI. I run offline on this computer.
I was made on September 25, 2026.
I don't have a body, feelings, a family, a boss, a job, a home address, a phone,
    an email address, a car or pets.          (one line in the file)
```

Example 21. mind_original/self.txt. The first line sets the name.

The chat prompt, the core (so "Lucy, make a file..." is read as a command) and the practice generators all read this file. Each training round runs `tools/rename_ai.py`, which writes the name into the chat data and the test copies. A brain trained under an older name keeps that name until a round retrains it. The lines about what the AI does not have become exact self-knowledge: "who is your boss?" gives `my boss -> I don't have one`.

15.4 Add Knowledge Without Training

Drop a package into `mind/memory/` and it is known before the next message (section 10.2). A single fact can also be taught in the chat, in a plain sentence ("A byte has 8 bits.") or with `/teach`, and a formula with `formula circle area(r) = pi * r ^ 2`. Fixed dates for day counting go in `mind/language/days.txt` as `name: month-day` (for example `my anniversary: 6-14`).

15.5 Teach the Core New Words

The words that ask for each action are in `mind/language/verbs.txt`, not in the C code. To make "peek at notes.txt" a read command, add the word to the read line:

```
read: read, open, show, display, look at, look in, look into, print, view, check, peek at | read
name.txt
```

Example 22. One line of mind/language/verbs.txt.

In the same way, a phrase added to a line of `mind/language/meanings.txt` teaches the work core another way of saying "it is still there" or "try again" (section 9.7).

15.6 Teach a Skill a New Fix

Know-how lines in `mind/skills/NAME.txt` are read before the skill's own (section 9.2), and the file is read again when it changes. The line below is one the computer skill already has: when a file has the name a new folder should get, it offers a free name instead. Written into `mind/skills/computer.txt`, a line like it changes or adds a fix without building anything:

```
# mind/skills/computer.txt
mkdir: if a file has that name: there is a file called {name 1} there already;
      ask mkdir {free {1}} = make the folder as "{free}" instead
```

Example 23. A know-how line for a cause (wrapped here; one line in the file).

15.7 Change What It Says

Every sentence the AI says on its own is a line in `mind/language/sayings.txt` (section 10.3). Change the words after the key and the next message uses them; add more lines with the same key and they are used in turn.

15.8 Write a Test

Core tests are plain text files and need no brain. `:world` sets the files that exist, `>` is a user message, `=` is a line the decision must contain, `!` is a word it must not contain, and `:reset` starts a new conversation:

```
# tests/my_test.txt
:world notes.txt work/
> move notes.txt into work
= move notes.txt to work
> what's in it?
= read work/notes.txt
:reset
> how many days until halloween
= days until halloween
! calc
```

Example 24. A small test file.

```
$ ./ai --sense-eval tests/my_test.txt --show
> move notes.txt into work
  do: move notes.txt to work -> moved
> what's in it?
  do: read work/notes.txt -> (text of work/notes.txt)
> how many days until halloween
  think: days until halloween -> 24 (saturday, october 31, 2026)
sense: 4 of 4 checks passed - ALL PASSED
```

Example 25. Running it (real output). "it" was resolved to the file that was just moved.

Whole conversations are tested through the real program with `tools/check_chat.py`. Each conversation starts with `===` and runs in a fresh copy of the folders; `~` words must be in the reply and `!` words must not:

```

=== the trash has one already: put it there under a free name
> Create a folder named 42
> Remove 42
> Create a folder named 42
> Remove 42
~ already has something called 42
~ 42 (2)
! Done, I made

$ python3 tools/check_chat.py tests/work_v1.txt
chat checks: 49 of 49 passed - ALL PASSED

```

Example 26. A conversation check and how it is run.

15.9 Train a Round

train_round.bat does a whole round with one click. Set HOURS (three to four is enough) and ROUND (a new number starts from the installed brain) at the top of the file. Each hour it regenerates and filters practice data, trains, tests and keeps the best checkpoint. At the end it installs the best brain as mind_original/start.web and packs trained_brain.zip with the scores:

```

hour 1: skill test 87.5%   your test 94.9%   unseen-chat perplexity 5.7
hour 2: skill test 88.6%   your test 94.9%   unseen-chat perplexity 6.2
hour 3: skill test 89.1%   your test 92.3%   unseen-chat perplexity 6.3

```

Example 27. scores.txt from a three-hour round.

Plain training on your own text is train.bat --minutes 120 mybooks\ chats.txt. Conversations use the format of section 7.1.

15.10 Add a New Kind of Neuron

Write a recipe (section 5.4), then load it and add neurons of that kind to every layer. They start silent, so the brain answers exactly as before until training puts them to work:

```

ai-train --recipe recipes/gated.rcp --add-recipe gated,32 --minutes 60 data
ai-train --grow-recipe gated --minutes 60 data # growth now adds gated neurons

```

Example 28. Adding 32 gated neurons per layer, then growing with them.

16. Roadmap

The rule for the next steps is **finish the core before scaling the data**. The core labels and filters all training data, so the thought formats must be frozen before a big-data phase. Steps marked ✓ are done.

D.R.O.N.E.

- 0. Measure where we are
 - head-to-head with a small local LLM (~1B): actions, honesty, memory, CPU speed
- 1. Finish the mind (core structure)
 - 1a. background loop + event queue: acts without a prompt (timers ✓)
 - 1b. tasks + solver: check → cause → fix → alternative → ask ✓
 - 1c. moral gate: within the user's intent, confirm irreversible steps (✓), no self-preservation, no deception or pressure [24]
 - 1d. one knowledge graph: profile + facts + concepts ("things and objects")
 - 1e. self-model: what it can do (✓), what it is doing (✓), its state
 - 1f. emotions as appraisal (OCC) [21]: colour the tone, never the correctness
 - 1g. putting skills together by itself, not only in the built-in ways
 - 1h. anticipation: notice what keeps coming back ("let me guess, sector C1?")
 - 1i. freeze the thought formats: the contract between core and voice
- 2. Understanding that generalizes: a small network trained on the core's own decisions, with the core still checking and vetoing
- 3. A bigger, better voice: GPU training, data quality, then big data
- 4. Learning from its own life: failures → practice → overnight training, kept only if held-out tests do not drop
- 5. Packs and bodies: knowledge/action packs, a body protocol, a 2D test world
- 6. Long term: NPC modes and character sheets, coding skill, the Worker D.R.O.N.E.

Example 29. The roadmap as a tree.

Repeated successes are already compiled into exact *reflexes*. The plan is to extend this into *core neurons*: exact building blocks that live inside the network, where the network learns when to call them. The idea is close to production compilation in ACT-R [25] and to tool use learned by language models [19].

17. References

- [1] A. Vaswani, N. Shazeer, N. Parmar, et al. *Attention Is All You Need*. NeurIPS, 2017.
- [2] B. Zhang, R. Sennrich. *Root Mean Square Layer Normalization*. NeurIPS, 2019.
- [3] J. Su, Y. Lu, S. Pan, et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864, 2021.
- [4] A. See, P. J. Liu, C. D. Manning. *Get To The Point: Summarization with Pointer-Generator Networks*. ACL, 2017.
- [5] O. Vinyals, M. Fortunato, N. Jaitly. *Pointer Networks*. NeurIPS, 2015.
- [6] T. Chen, I. Goodfellow, J. Shlens. *Net2Net: Accelerating Learning via Knowledge Transfer*. ICLR, 2016.
- [7] D. P. Kingma, J. Ba. *Adam: A Method for Stochastic Optimization*. ICLR, 2015.
- [8] S. Welleck, I. Kulikov, S. Roller, et al. *Neural Text Generation with Unlikelihood Training*. ICLR, 2020.
- [9] S. Hu, Y. Tu, X. Han, et al. *MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies*. arXiv:2404.06395, 2024.
- [10] A. Hägele, E. Bakouch, A. Kosson, et al. *Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations*. NeurIPS, 2024.
- [11] A. Holtzman, J. Buys, L. Du, et al. *The Curious Case of Neural Text Degeneration*. ICLR, 2020.
- [12] Y. Wu, M. Schuster, Z. Chen, et al. *Google's Neural Machine Translation System*. arXiv:1609.08144, 2016.
- [13] O. Press, L. Wolf. *Using the Output Embedding to Improve Language Models*. EACL, 2017.
- [14] S. Zhang, E. Dinan, J. Urbanek, et al. *Personalizing Dialogue Agents: I Have a Dog, Do You Have Pets Too?* ACL, 2018.
- [15] Y. Li, H. Su, X. Shen, et al. *DailyDialog: A Manually Labelled Multi-turn Dialogue Dataset*. IJCNLP, 2017.
- [16] K. Gopalakrishnan, B. Hedayatnia, Q. Chen, et al. *Topical-Chat: Towards Knowledge-Grounded Open-Domain Conversations*. Interspeech, 2019.

- [17] P. Rajpurkar, J. Zhang, K. Lopyrev, P. Liang. *SQuAD: 100,000+ Questions for Machine Comprehension of Text*. EMNLP, 2016.
- [18] R. Eldan, Y. Li. *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* arXiv:2305.07759, 2023.
- [19] T. Schick, J. Dwivedi-Yu, R. Dessì, et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS, 2023.
- [20] J. Li, M. Galley, C. Brockett, J. Gao, B. Dolan. *A Diversity-Promoting Objective Function for Neural Conversation Models*. NAACL, 2016.
- [21] A. Ortony, G. L. Clore, A. Collins. *The Cognitive Structure of Emotions*. Cambridge University Press, 1988.
- [22] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*. (AVX2, FMA and instruction encoding.)
- [23] The Khronos Group. *SPIR-V Specification and Vulkan Specification*.
- [24] Anthropic. *Agentic Misalignment: How LLMs Could Be Insider Threats*. 2025.
- [25] J. R. Anderson. *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press, 2007.
- [26] N. Muennighoff, A. M. Rush, B. Barak, et al. *Scaling Data-Constrained Language Models*. NeurIPS, 2023.

Appendix A. Glossary

Table 15. *Terms used in this document.*

Term	Meaning
Drone	An AI built on the D.R.O.N.E. engine, with its own name (<code>mind_original/self.txt</code>), brain and memory. The first one is Lucy.
Core	The exact C part that understands and decides (<code>SENSE.c</code> , <code>KNOWLEDGE.c</code> , <code>CALCULATOR.c</code> , <code>SOLVE.c</code>).
Work core	The exact part that runs each task, checks it and fixes what went wrong (<code>WORK.c</code>).
Task	One change the user asked for, with its steps, its cause and the fixes tried.
Skill	A plug-in library that adds actions, with its own know-how.
Know-how	Plain lines that say what an action makes true and what to try when it fails.
Knowledge package	A plain-text file of things the AI knows (<code>mind/memory/*.mem</code>).
Saying	One of the AI's own sentences, kept in <code>mind/language/sayings.txt</code> .
Voice / brain	The neural network that puts decisions into words.
Thought line	A fixed-format decision from the core, such as <code>did send_email -> no</code> .
Stub	The few bytes of machine code that make up one neuron.
Organ	Shared AVX2/FMA code that does the math for one kind of neuron in one area.
Recipe	A text description of a new neuron type, compiled by the engine.
Reflex	A repeated success compiled into an exact switch in machine code.
Round	One training run of three to six hours, tested every hour.
Kit	A zip with the engine, data tools and scripts.
Sense filter	The core run over practice data to drop examples it disagrees with.